Learn Practice Improve Grow

Python Sets

A Collection of Unique Elements

Small Steps Big Skills!

Keep Practicing Python!

What is a Set?

- A **set** is an **unordered** collection of **unique elements** in Python.
- Elements in a set do not allow duplicates.
- Sets are **mutable** (can be modified).
- Sets are written using curly braces { } or the **set()** function.
- Elements can be of different data types.

Creating a Set

```
# Using curly braces
s1 = {1, 2, 3, 4}
print(s1)

# Using set() function
s2 = set([10, 20, 30])
print(s2)

# Mixed data types
s3 = {1, "hello", 3.5}
print(s3)
```

Output:

```
{1, 2, 3, 4}
{10, 20, 30}
{1, 3.5, 'hello'}
```

Important Points

- Elements are unique (duplicates are removed).
- Sets are unordered (no indexing).
- Sets are mutable (can add or remove elements).
- Sets cannot be accessed using index (s[0] will give an error).
- Allows different data types.
- Useful for mathematical operations like union, intersection, difference, etc.

Set Operations

Operation	Symbol / Method	Example	Result
Union	| or union()	{1, 2} | {2, 3}	{1, 2, 3}
Intersection	& or intersection()	{1, 2} & {2, 3}	{2}
Difference	- or difference()	{1, 2} - {2, 3}	{1}
Symmetric Difference	^ or symmetric_difference()	{1, 2} ^ {2, 3}	{1, 3}

Adding and Removing Elements

```
s = {1, 2, 3}
s.add(4)       # Add element
print(s)
s.remove(2)    # Remove element (error if not present)
print(s)
s.discard(10)  # Removes element (no error if not present)
s.clear()      # Remove all elements
print(s)
```

Practice Question

What will be the output of the following code?

```
s = {10, 20, 20, 30, 40}
s.add(50)
s.remove(30)
print(s)
```

A {10, 20, 30, 40}

B {10, 20, 40, 50}

C {20, 30, 40, 50}

D {10, 20, 30, 40, 50}

Answer: B

Explanation:

Initially, duplicates are removed:

```
s = {10, 20, 30, 40}
```

After `s.add(50)`:

```
s = {10, 20, 30, 40, 50}
```

After `s.remove(30)`:

```
s = {10, 20, 40, 50}
```

So, the output is {10, 20, 40, 50}.

Key Takeaways

- Sets store only unique elements.
- Sets are unordered.
- No indexing (s[0] gives error).
- Use add(), remove(), discard(), clear() to modify sets.
- Support mathematical operations (union, intersection, difference, symmetric difference).
- Useful in real-world problems like finding common items, removing duplicates, etc.

"Practice Today, Stronger Tomorrow!"

By Satyarthi

Learn Practice Grow

Python Set Operations

Small Steps Big Skills!

Perform Operations on Sets

1. Union (| or union())

Returns all unique elements from both sets.

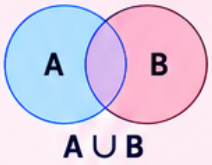
A | B or A.union(B)

Example Code:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)   # or A.union(B)
```

Output:

```
{1, 2, 3, 4, 5}
```



2. Intersection (& or intersection())

Returns elements that are common in both sets.

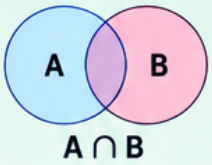
A & B or A.intersection(B)

Example Code:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A & B)  # or A.intersection(B)
```

Output:

```
{3}
```



3. Difference (- or difference())

Returns elements in **A** that are not in **B**.

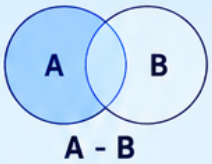
A - B or A.difference(B)

Example Code:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A - B)  # or A.difference(B)
```

Output:

```
{1, 2}
```



4. Reverse Difference

Returns elements in **B** that are not in **A**.

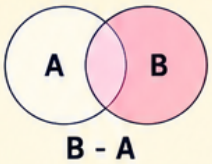
B - A or B.difference(A)

Example Code:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(B - A)  # or B.difference(A)
```

Output:

```
{4, 5}
```



5. Symmetric Difference (^ or symmetric_difference())

Returns elements that are in either set, but not in both.

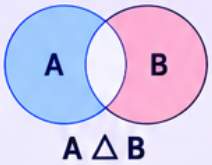
A ^ B or A.symmetric_difference(B)

Example Code:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A ^ B)  # or A.symmetric_difference(B)
```

Output:

```
{1, 2, 4, 5}
```



6. Adding an Element (add())

Adds a single element to the set.

S.add(x)

Example Code:

```
S = {1, 2, 3}
S.add(4)
print(S)
```

Output:

```
{1, 2, 3, 4}
```

Adds only one element. Duplicates are ignored.

7. Removing an Element (remove())

Removes a specific element (fom the set (raises an error if not present).

S.remove(x)

Example Code:

```
S = {1, 2, 3}
S.remove(2)
print(S)
```

Output:

```
{1, 3}
```

If the element is not present, it raises a **KeyError**.

8. Discard an Element (discard())

Removes a specific element (fom the set (no error if not present).

S.discard(x)

Example Code:

```
S = {1, 2, 3}
S.discard(5)
print(S)
```

Output:

```
{1, 2, 3}
```

If the element is not present, no error occurs.

9. Clear Set (clear())

Removes all elements from the set.

S.clear()

Example Code:

```
S = {1, 2, 3}
S.clear()
print(S)
```

Output:

```
set()
```

Makes the set empty.

10. Length of a Set (len())

Returns the number of elements in the set.

len(S)

Example Code:

```
S = {10, 20, 30, 40}
print(len(S))
```

Output:

```
4
```

Counts only unique elements.

Consistent Practice Builds Strong Python Skills!"

By Satyarthi

Learn Practice Improve Grow

Python Lists & Tuples

Practice Question

Topic: Lists vs Tuples

Small Steps Big Skills!

Question

What will be the output of the following code?

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
my_list[0] = 100
print(my_list)
print(my_tuple)
```

A
```
[100, 2, 3]
(1, 2, 3)
```

B
```
[1, 2, 3]
(100, 2, 3)
```

C
```
[100, 2, 3]
(100, 2, 3)
```

D
Error: 'tuple' object does not support item assignment

Lists are mutable, Tuples are immutable!

Answer: A

Explanation:

- Lists are mutable, so the first element can be changed.
- Tuples are immutable, so the value remains unchanged.
- The output will be:

```
[100, 2, 3]
(1, 2, 3)
```

Visual Representation

List (Mutable)

my_list = [1, 2, 3]

0	1	2
1	2	3

my_list[0] = 100

0	1	2
100	2	3

Tuple (Immutable)

my_tuple = (1, 2, 3)

0	1	2
1	2	3

✗ Cannot change value (Immutable)

Key Points

- Lists are mutable (can be changed).
- Tuples are immutable (cannot be changed).
- Both can store similar data types.
- Use lists when you need to modify data.
- Use tuples when you want data to remain constant (e.g., coordinates, fixed records).

Remember

- List → [] (square brackets)
- Tuple → () (parentheses)
- List methods: append(), remove(), etc.
- Tuple methods: count(), index(), etc.
- Try changing values in both and see the difference!

"Practice Today, Build Tomorrow!"

By Satyarthi

Learn Practice Improve Grow

Python Sets

Practice Question

Test Your Understanding

Small Steps Big Skills!

Question

What will be the output of the following code?

```
s = {1, 2, 3, 4}
s.add(4)
s.add(5)
s.remove(2)
print(s)
```

A {1, 3, 4, 5}

B {1, 2, 3, 4, 5}

C {1, 2, 4, 5}

D {3, 4, 5}

Think Logically Apply Concepts Choose the Correct Output!

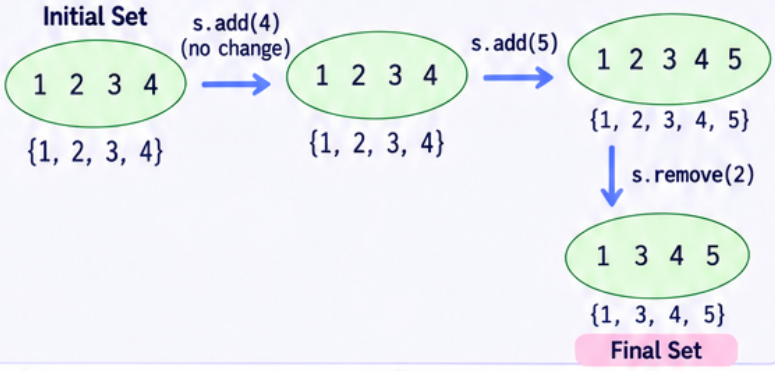
Answer: A

Explanation:

- **Initial set:**
  `s = {1, 2, 3, 4}`
- **s.add(4):**
  4 is already present, so no change
  `s = {1, 2, 3, 4}`
- **s.add(5):**
  `s = {1, 2, 3, 4, 5}`
- **s.remove(2):**
  `s = {1, 3, 4, 5}`
- Finally, `print(s)` will output:

```
{1, 3, 4, 5}
```

Visual Representation



Key Points

- A set is an unordered collection of unique elements.
- Duplicate elements are ignored.
- Use add() to insert an element.
- Use remove() to delete an element (raises error if not present).
- Sets are mutable.
- The order of elements in the output may vary.

Related Functions

- add() – Add an element
- remove() – Remove an element (error if not present)
- discard() – Remove an element (no error if not present)
- union() – Combine two sets
- intersection() – Common elements
- difference() – Elements in A not in B
- clear() – Remove all elements
- len() – Number of elements

Consistent Practice Builds Strong Python Skills!

By Satyarthi

Learn Practice Improve Grow

Python Sets

Practice Question

Topic: Set Operations

Small Steps Big Skills!

Keep Practicing Python!

Question

What will be the output of the following code?

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
C = A | B       # union of A and B
D = A & B       # intersection of A and B
E = A - B       # elements in A but not in B
F = B - A       # elements in B but not in A
print(C, D, E, F)
```

Apply Set Concepts and Find the Output!

A `{1, 2, 3, 4, 5, 6} {3, 4} {1, 2} {5, 6}`

B `{1, 2, 3, 4} {3, 4} {1, 2} {5, 6}`

C `{1, 2, 3, 4, 5, 6} {1, 2} {3, 4} {5, 6}`

D `{3, 4} {1, 2, 5, 6} {1, 2} {5, 6}`

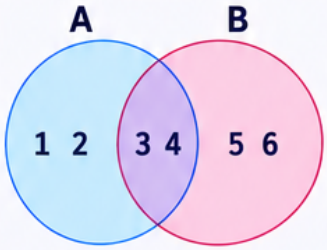
Answer: A

Explanation:

- **A | B** gives the union of both sets (all unique elements).
  `C = {1, 2, 3, 4, 5, 6}`
- **A & B** gives the common elements (intersection).
  `D = {3, 4}`
- **A - B** gives elements in A but not in B.
  `E = {1, 2}`
- **B - A** gives elements in B but not in A.
  `F = {5, 6}`
- **So, the output will be:**

```
{1, 2, 3, 4, 5, 6} {3, 4} {1, 2} {5, 6}
```

Set Visualization



A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
A | B = {1, 2, 3, 4, 5, 6}
A & B = {3, 4}
A - B = {1, 2}
B - A = {5, 6}

Key Takeaways

- Sets store unique elements (no duplicates).
- Common set operations: union (|), intersection (&), difference (-).
- Sets are unordered, so output order may vary.
- Use sets when you need to work with unique data and perform mathematical operations.
- Very useful in real-world problems like finding common items, removing duplicates, etc.

"Practice Today, Stronger Tomorrow!"

By Satyarthi

Learn Practice Improve Grow

Python Sets

Practice Question

Test Your Knowledge

Small Steps Big Skills!

Keep Practicing Python!

Question

What will be the output of the following code?

```
A = {1, 2, 3, 4}
B = {3, 4, 4, 5}
A.update(B)
B.add(6)
A.remove(2)
print(A)
print(B)
```

Understand Sets Solve Better!

A

```
{1, 2, 3, 4, 5}
{3, 4, 5, 6}
```

B

```
{1, 3, 4, 5}
{3, 4, 5, 6}
```

C

```
{1, 3, 4, 5, 6}
{3, 4, 6}
```

D

```
{1, 3, 4, 6}
{3, 4, 5, 6}
```

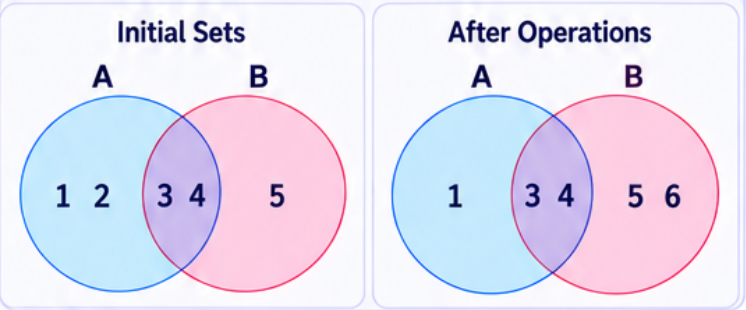
Answer: B

Explanation:

- **Initially:**
  `A = {1, 2, 3, 4}`
  `B = {3, 4, 5}` (duplicate 4 is ignored)
- **A.update(B):**
  A becomes `{1, 2, 3, 4, 5}`
  (adds all elements from B)
- **B.add(6):**
  B becomes `{3, 4, 5, 6}`
- **A.remove(2):**
  2 is removed from A
  A becomes `{1, 3, 4, 5}`
- **Final output:**

```
{1, 3, 4, 5}
{3, 4, 5, 6}
```

Step-by-Step Visualization



Key Takeaways

- Sets store unique elements (no duplicates).
- update() adds all elements from another set.
- add() adds a single element.
- remove() removes an element (raises error if not present).
- Set elements are unordered (output order may vary).

Consistent Practice Builds Strong Python Skills!

By Satyarthi

Learn Practice Improve Grow

Python Dictionaries

Store Data in Key-Value Pairs

A Powerful & Flexible Data Structure

Small Steps Big Skills!

What is a Dictionary?

- A dictionary in Python is a collection of key-value pairs.
- It is unordered, mutable and allows unique keys.
- Keys can be strings, numbers or tuples (immutable).
- Values can be of any data type.

Syntax:

```
dict_name = {key1: value1,
             key2: value2,
             ...}
```

Creating a Dictionary

```
>>> student = {"name": "Aman",
               "age": 19,
               "course": "BPT"}
>>> print(student)
{'name': 'Aman', 'age': 19,
'course': 'BPT'}
```

Accessing Values

- Use keys to access values.

```
>>> student = {"name": "Aman",
               "age": 19}
>>> print(student["name"])
Aman
>>> print(student["age"])
19
```

Adding or Updating Items

- Use **key = value** to add a new item or update an existing one.

```
>>> student = {"name": "Aman"}
>>> student["age"] = 19   # add
>>> student["name"] = "Sundram"
    # update
>>> print(student)
{'name': 'Sundram', 'age': 19}
```

Removing Items

- **pop()** – removes item by key
- **del** – removes item by key
- **clear()** – removes all items

```
>>> student = {"name": "Aman",
               "age": 19}
>>> student.pop("age")
19
>>> print(student)
{'name': 'Aman'}
>>> del student["name"]
>>> print(student)
{}
```

Common Dictionary Methods

Method	Description
get(key)	Returns value (no error if key not found)
keys()	Returns all keys
values()	Returns all values
items()	Returns all key-value pairs
update()	Updates with another dictionary
pop(key)	Removes and returns value for the key
clear()	Removes all items
len()	Returns number of items

Looping Through a Dictionary

- You can iterate using keys, values or items.

```
>>> student = {"name": "Aman",
               "age": 19,
               "course": "BPT"}
>>> for key in student:       # keys
...     print(key, ":", student[key])
name : Aman
age : 19
course : BPT
```

Nested Dictionary

- A dictionary can contain another dictionary.

```
>>> students = {
...     "s1": {"name": "Aman", "age": 19},
...     "s2": {"name": "Riya", "age": 20}
... }
>>> print(students["s1"]["name"])
Aman
```

Example: Student Marks

```
>>> marks = {"Physics": 85,
             "Chemistry": 78,
             "Biology": 92}
>>> print(marks["Biology"])
92
>>> marks["Maths"] = 88
>>> print(marks)
{'Physics': 85, 'Chemistry': 78,
'Biology': 92, 'Maths': 88}
```

Practice Question

Create a dictionary with the following details and perform the given tasks.

```
student = {"name": "Rohit", "age": 18, "course": "BSc"}
```

1. Add a new key "city" with value "Patna".
2. Update the age to 19.
3. Remove the "course" key.
4. Print the final dictionary.

Expected Output

```
{'name': 'Rohit',
'age': 19,
'city': 'Patna'}
```

Key Takeaways

- Dictionary = key-value pairs.
- Keys are unique and immutable.
- Values can be any data type.
- Dictionaries are mutable.
- Useful for real-world data (e.g., student records, contact lists, configuration).

"Practice Today, Build Tomorrow!"

By Satyarthi

Learn
Practice
Improve
Grow

Python Dictionaries

Two Important Concepts

Small Steps Big Skills!
Keep Practicing Python!

1 Creating a Dictionary

- A dictionary is a collection of key-value pairs written inside curly braces { }.
- Each item is in the form key : value, separated by commas.
- Keys must be unique and can be of any immutable type (string, number, tuple).
- Values can be of any data type.

Syntax:

```
dictionary_name = {key1: value1,
                   key2: value2,
                   ...}
```

Example:

```
# Creating a dictionary
student = {
    "name": "Aman",
    "age": 19,
    "course": "BPT"
}
print(student)
```

Output:

```
{'name': 'Aman', 'age': 19,
'course': 'BPT'}
```

2 Using dict() Constructor

- You can also create a dictionary using the built-in dict() function.
- Pass key=value pairs inside the parentheses, separated by commas.
- It is an alternative way to create a dictionary and is useful when creating a dictionary programmatically.

Syntax:

```
dictionary_name = dict(key1=value1,
                       key2=value2,
                       ...}
```

Example:

```
# Creating a dictionary using dict()
student = dict(name="Rohit",
               age=18,
               course="BSc")

print(student)
```

Output:

```
{'name': 'Rohit', 'age': 18,
'course': 'BSc'}
```

Consistent Practice Builds Strong Python Skills!

By Satyarthi

Learn Practice Improve Grow

Python Dictionaries

Next 2 Important Concepts

Small Steps Big Skills!

Keep Practicing Python!

3 Accessing Dictionary Items

You can access values in a dictionary using the key. If the key is not present, it will raise a **KeyError**.

Syntax:

```
dictionary_name[key]
# or
dictionary_name.get(key)  # safer method
```

Example:

```
# Creating a dictionary
student = {
    "name": "Aman",
    "age": 19,
    "course": "BPT"
}
# Accessing values
print(student["name"])      # Aman
print(student["age"])       # 19
print(student.get("course"))  # BPT

# Trying to access a key that doesn't exist
print(student.get("city", "Not Found"))
# Not Found
```

Output:

```
Aman
19
BPT
Not Found
```

4 Updating and Adding Items

You can change the value of an existing key (update), or add a new key-value pair to a dictionary.

Syntax:

```
dictionary_name[key] = value  # update
dictionary_name[new_key] = value  # add
```

Example:

```
# Creating a dictionary
student = {
    "name": "Aman",
    "age": 19,
    "course": "BPT"
}

# Updating an existing value
student["age"] = 20
# age updated from 19 to 20

# Adding a new key-value pair
student["city"] = "Patna"
# new key added

print(student)
```

Output:

```
{'name': 'Aman', 'age': 20,
'course': 'BPT', 'city': 'Patna'}
```

Key Takeaways

- Use dictionary_name[key] to access a value.
- Use get(key, default) to avoid errors.
- You can update the value of an existing key.
- You can add new key-value pairs easily.
- Keys must be unique.
- Dictionaries are mutable (can be changed).

Practice Build Concepts Do Better!

By Satyarthi

Learn Practice Improve Grow

Python Dictionaries

Next 2 Important Concepts

Small Steps Big Skills!

5 Dictionary Methods

Python provides several built-in methods to work with dictionaries, such as `keys()`, `values()`, `items()`, `get()`, etc.

Method	Description
`keys()`	Returns all the keys
`values()`	Returns all the values
`items()`	Returns all key-value pairs (as tuples)
`get(key)`	Returns value for the key (if key not found, returns None or a default value)
`update()`	Updates the dictionary with another dictionary
`pop(key)`	Removes the specified key and returns its value
`clear()`	Removes all items
`len()`	Returns the number of items

Example:

```
student = {"name": "Aman",
           "age": 19,
           "course": "BPT"}
print(student.keys())
print(student.values())
print(student.items())
print(student.get("age"))
```

Output:

```
dict_keys(['name', 'age', 'course'])
dict_values(['Aman', 19, 'BPT'])
dict_items([('name', 'Aman'), ('age', 19),
('course', 'BPT')])
19
```

6 Looping Through a Dictionary

You can iterate through a dictionary using a for loop to access keys, values, or both keys and values.

1. Looping through keys

```
student = {"name": "Aman",
           "age": 19,
           "course": "BPT"}

for key in student:
    print(key)
```

Output:

```
name
age
course
```

2. Looping through values

```
for value in student.values():
    print(value)
```

Output:

```
Aman
19
BPT
```

3. Looping through key-value pairs

```
for key, value in student.items():
    print(key, ":", value)
```

Output:

```
name : Aman
age : 19
course : BPT
```

4. Using items() in a different way

```
for item in student.items():
    print(item)
```

Output:

```
('name', 'Aman')
('age', 19)
('course', 'BPT')
```

Key Takeaways

- Use built-in methods to easily work with dictionaries.
- You can loop through keys, values or both.
- `items()` returns key-value pairs as tuples.
- Dictionaries are flexible and very useful for real-world data (e.g., student records, contact lists, configuration).
- Practice these concepts with examples to strengthen your understanding.

Practice Today, Build a Better Tomorrow!

By Satyarthi

Code Practice Grow

Python String Methods

Practice Question 3

Small Practice Big Improvement!

Question 3

Topic: **replace()** method

What will be the output of the following code?

```
s = "I love Python"
print(s.replace("Python", "Java"))
```

A 'I love Java'

B 'I Love Java'

C 'I love python'

D 'I love Python Java'

"Same Method, New Possibilities!"

Answer: A

Explanation:

The **replace()** method returns a new string in which all occurrences of the specified substring are replaced with another substring.

Here, **"Python"** is replaced by **"Java"**, so the output is **'I love Java'**.

The original string remains unchanged.

Key Takeaway

- **replace()** replaces all occurrences of a substring.
- It is case-sensitive.
- It does not modify the original string (strings are immutable in Python).

```
s = "apple apple"
print(s.replace("apple", "orange"))
# Output: orange orange
```

Practice Today, Stronger Tomorrow!

By Satyarthi

Learn Practice Improve Grow

Python Dictionaries

Next 2 Important Concepts

Small Steps Big Skills!

Keep Practicing Python!

7 Removing Items from a Dictionary

You can remove an item from a dictionary using `pop()`, `popitem()`, `del`, or `clear()`.
These methods help you delete a specific key, the last item, or all items.

1. Using pop(key)

Removes the specified key and returns its value.
If the key is not found, it raises a `KeyError`.

```
student = {"name": "Aman",
"age": 19, "course": "BPT"}
removed = student.pop("age")
print(removed)
print(student)
```

Output:

```
19
{'name': 'Aman',
'course': 'BPT'}
```

2. Using popitem()

Removes and returns the last inserted key-value pair (as a tuple).

```
student = {"name": "Aman",
"age": 19}
item = student.popitem()
print(item)
print(student)
```

Output:

```
('age', 19)
{'name': 'Aman'}
```

3. Using del

Removes the specified key (does not return the value).

```
student = {"name": "Aman",
           "age": 19}
del student["name"]
print(student)
```

Output:

```
{'age': 19}
```

4. Using clear()

Removes all items from the dictionary.

```
student = {"name": "Aman",
           "age": 19}
student.clear()
print(student)
```

Output:

```
{}
```

8 Dictionary Comprehension

You can create a dictionary using dictionary comprehension, similar to list comprehension.
It provides a short and clean way to create dictionaries based on a condition or expression.

1. Creating a dictionary with squares (1 to 5)

```
squares = {x: x**2 for x in range(1, 6)}
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

2. Creating a dictionary with a condition

```
even_squares = {x: x**2 for x in range(1, 11)
                if x % 2 == 0}
print(even_squares)
```

Output:

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

3. Creating a dictionary from two lists

```
keys = ["name", "age", "city"]
values = ["Aman", 19, "Patna"]
student = {keys[i]: values[i] for i in range(len(keys))}
print(student)
```

Output:

```
{'name': 'Aman', 'age': 19, 'city': 'Patna'}
```

Key Takeaways

- Use pop(), popitem(), del, or clear() to remove items.
- Dictionary comprehension helps you create dictionaries in a single line.
- You can use conditions inside dictionary comprehension.
- Dictionaries are flexible and useful for real-world data handling.
- Practice with different examples to strengthen your concepts.

Consistent Practice Builds Strong Python Skills!

By Satyarthi

Learn Practice Improve Grow

Python Dictionaries

Practice Questions

Small Steps Big Skills!

Keep Practicing Python!

1 Question 1

Consider the following dictionary:

```
student = {
    "name": "Aman",
    "age": 19,
    "course": "BPT"
}
```

What will be the output of the following code?

```
print(student.get("city", "Not Found"))
```

A. None

B. Not Found

C. Error

D. "city"

Concept Revision:

- The `get()` method returns the value for a key if it exists.
- If the key is not found, it returns the specified default value (here `"Not Found"`).
- It does not raise an error.

2 Question 2

What will be the output of the following code?

```
marks = {"Physics": 85, "Chemistry": 78}
marks["Biology"] = 92
marks["Chemistry"] = 80
del marks["Physics"]
print(marks)
```

A. `{'Physics': 85, 'Chemistry': 78, 'Biology': 92}`

B. `{'Chemistry': 80, 'Biology': 92}`

C. `{'Physics': 85, 'Chemistry': 80, 'Biology': 92}`

D. `{'Chemistry': 78, 'Biology': 92}`

Concept Revision:

- You can add a new key-value pair using `dict[key] = value`.
- If a key already exists, its value gets updated.
- You can remove a key using `del dict[key]`.
- The final dictionary contains the remaining key-value pairs.

Consistent Practice Builds Strong Python Skills!

By Satyarthi